

UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
FACULTAD DE INGENIERÍA

CINESPOT

Análisis de modelos de comunicación

JOSIAS PEREZ, JORGE COLAMARCO

13 de octubre de 2026

Descripción

CineSpot es un proyecto académico que se desarrollará en Python para simular una plataforma de reserva de asientos en una sala de cine. Permitirá consultar la disponibilidad de los asientos y solicitar la reserva de uno libre para una función determinada. Antes de confirmar cada solicitud, el sistema deberá validar el asiento y actualizar su estado. El problema que se analiza es cómo mantener una disponibilidad confiable cuando varias solicitudes coinciden, de modo que un asiento no pueda asignarse más de una vez para la misma función.

Análisis

Para analizar la coordinación, se considera que distintas solicitudes pueden atenderse al mismo tiempo. Cada solicitud debe comprobar el estado del asiento elegido y, si sigue disponible, reservarlo. Si dos solicitudes coinciden sobre el mismo asiento, ambas podrían leerlo como libre antes de que alguna registre el cambio. Por eso, la comprobación y la actualización deben coordinarse como una sola operación.

Unidades de ejecución y recurso compartido

Cada unidad de ejecución representa un trabajador que procesa la solicitud de un usuario. El recurso compartido es el mapa de asientos de una función: registra cada asiento y su estado, disponible o reservado. El identificador de la función forma parte del registro, porque un mismo asiento puede estar disponible en una función y reservado en otra. Los trabajadores deben cooperar para consultar y cambiar ese estado sin duplicar asignaciones.

Comparación de modelos

En memoria compartida, varios hilos acceden a datos comunes y los accesos concurrentes que pueden entrar en conflicto requieren sincronización (OpenMP Architecture Review Board, 2021). En el paso de mensajes, los procesos intercambian solicitudes y respuestas de forma explícita, por medio de comunicaciones entre procesos (Message Passing Interface Forum, 2023). La tabla 1 muestra cómo aplicar ambos enfoques al control de asientos de CineSpot.

Tabla 1. Comparación de los modelos de comunicación en CineSpot

Modelo	Aplicación en CineSpot	Ventajas y límites
Memoria compartida	Varios hilos atienden solicitudes y consultan el mismo mapa de asientos. Un threading.Lock protege la sección que verifica que el asiento esté disponible y lo cambia a reservado; Python documenta este bloqueo para sincronizar hilos (Python Software Foundation, s. f.-b).	Es directo para una simulación en un solo proceso y evita intercambiar mensajes por cada solicitud. Sin el bloqueo alrededor de la comprobación y el cambio, dos hilos podrían reservar el mismo asiento.

Modelo	Aplicación en CineSpot	Ventajas y límites
Paso de mensajes	Cada proceso trabajador envía función, asiento y usuario por una multiprocessing.Queue a un coordinador. El coordinador es dueño del mapa, procesa una solicitud, valida el estado, lo actualiza si corresponde y devuelve una respuesta. Python ofrece colas para intercambiar datos entre procesos (Python Software Foundation, s. f.-a).	El mapa tiene un único responsable y los trabajadores no escriben directamente en él. A cambio, hay que gestionar mensajes y respuestas; el coordinador también puede convertirse en un punto de espera.

Conclusión del análisis

Para el alcance académico actual, se recomienda memoria compartida con hilos. La simulación mantiene el inventario de una función en una estructura común dentro de la aplicación; un bloqueo breve alrededor de la comprobación y la reserva permite proteger la operación que evita duplicados. Este diseño es directo y requiere menos coordinación adicional que una cola y un proceso administrador. El paso de mensajes sería conveniente si CineSpot separara sus trabajadores en procesos o servicios independientes, pues un coordinador podría conservar la autoridad sobre el mapa de asientos. Para esta simulación, esa separación agregaría manejo de mensajes y respuestas que no resulta necesario. La recomendación se basa en el alcance descrito; no se afirma una ventaja de rendimiento medida.

Bibliografía

Message Passing Interface Forum. (2023). MPI: A message-passing interface standard version 4.1. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report/mpi41-report.htm

OpenMP Architecture Review Board. (2021). OpenMP application programming interface specification version 5.2. https://www.openmp.org/spec-html/5.2/openmp.html

Python Software Foundation. (s. f.-a). multiprocessing - Process-based parallelism. Python 3.14 documentation. Recuperado el 13 de octubre de 2026 de https://docs.python.org/3.14/library/multiprocessing.html

Python Software Foundation. (s. f.-b). threading - Thread-based parallelism. Python 3.14 documentation. Recuperado el 13 de octubre de 2026 de https://docs.python.org/3.14/library/threading.html